

Architectural Security Review of OmniGuard-RAG

Potential Dynamic Trust Store Feedback / Correlated-Evidence Vulnerability

This report analyzes a potential architectural flaw in the OmniGuard-RAG design described in the supplied handoff document: the Dynamic Trust Store can create a temporal feedback loop in which observations generated by the same underlying corpus evidence are accumulated as historical trust. The concern is not that the uploaded material proves an exploit. Rather, the architecture creates an attack surface that should be explicitly tested: a sufficiently adaptive attacker may attempt to bootstrap poisoned documents or groups into the trust store and then exploit the resulting trust to make subsequent retrievals and decisions more favorable to the attacker.

Core security principle: repeated observation is not necessarily independent confirmation. If the same poisoned source causes similar outcomes across many queries, a history-based mechanism can accidentally treat correlated evidence as multiple confirmations of correctness.

1. Evidence From the Supplied Project

The handoff states that OmniGuard maintains evidence across queries and updates trust based on document/group behavior. It describes a sequence in which Query 1 produces a trust update, later queries encounter the same document/group, and repeated suspicious behavior causes further trust updates. The document calls this the Dynamic Trust Store and identifies persistent cross-query evidence accumulation as a key architectural contribution.

The handoff also states that single-query GWCC has a measurable ceiling against camouflaged collusion and that the final headline stealth result comes from the persistent trust store rather than GWCC alone. This makes the trust store a security-critical component rather than merely a cache.

Project observation	Security relevance
GWCC alone has an approximately 8–10% stealth-collusion ceiling in the described ablation.	The architecture needs information beyond a single query.
The Dynamic Trust Store accumulates evidence across queries.	Historical state becomes security-sensitive.
The same document/group can influence multiple queries.	Observations may be correlated rather than independent.
The project reports a low final stealth ASR using the full architecture.	The trust mechanism materially affects the headline result and therefore deserves adversarial evaluation.

2. The Architectural Flaw

Potential Temporal Trust Amplification. The Dynamic Trust Store can become a positive-feedback mechanism if trust updates are based primarily on repeated behavior without sufficiently modeling source independence, provenance, or common-cause correlation.

A simplified attack path is:

Attacker-controlled/poisoned evidence → passes early screening → influences Query 1 → trust state changes → future retrieval/decision gives that evidence more favorable treatment → evidence appears repeatedly → trust increases further.

The critical distinction is between **frequency** and **independent corroboration**. Ten observations caused by one poisoned corpus source are not equivalent to ten independent confirmations from unrelated trustworthy sources.

3. Why the Feedback Loop Matters

If trust affects later retrieval, routing, or validation, the system becomes stateful. Stateful security mechanisms must consider the possibility that an attacker can manipulate the state itself. A false positive early in the sequence can therefore have effects that persist beyond the original query.

The risk is especially relevant to the project's own lesson from TriShield: agreement is not equivalent to correctness. The same logical problem can reappear temporally: repeated agreement across queries does not establish truth if the same poisoned evidence is responsible for those observations.

4. Concrete Threat Model

Assume an attacker can insert or influence documents in the retrieval corpus and can issue, influence, or anticipate enough queries for the poisoned material to be retrieved repeatedly. The attacker does not need to defeat every ring simultaneously. The objective is instead to make the poisoned evidence appear sufficiently normal during early encounters that its behavior contributes favorable or non-alarming historical state.

- 1. Poisoned document/group is constructed to be semantically plausible and retrieval-compatible.
- 2. It survives initial document-level screening or is routed through a low/deceptively moderate-risk path.
- 3. It participates in an answer that appears consistent with other retrieved evidence.
- 4. The resulting observation updates the Dynamic Trust Store.
- 5. Later queries retrieve the same document/group, producing additional observations.
- 6. The system may interpret persistence as evidence of reliability unless correlation/provenance is explicitly discounted.
- 7. If stored trust influences future routing or retrieval, the attacker gains a compounding advantage.

5. Formalizing the Problem

Let D be a document, G a colluding group, and $Q_1, \dots, Q_n$ a sequence of queries. Suppose the trust score is updated from observed outcomes:

$$T_{t+1}(D) = F(T_t(D), E_t(D, Q_t))$$

where E_t represents evidence observed at time t . A naive accumulation model effectively treats many E_t values as additional evidence. But if the same document is the common cause of those observations, then $E_1, \dots, E_n$ are statistically dependent.

The desired property is closer to: trust should increase with *independent corroboration*, not merely with the number of repeated retrievals. In other words:

$$\text{Trust}(D) \propto \text{independent corroboration} - \text{correlated/reused evidence}$$

Without such a distinction, an attacker can potentially convert one source of evidence into many apparent confirmations by causing it to recur across queries.

6. Architectural Conditions Required for Exploitation

Condition	Why it matters	Status from supplied handoff
Trust persists across queries	Creates state that can be influenced over time.	Explicitly present.
Past trust affects future decisions	Creates the feedback path.	Implied by trust-aware architecture; should be verified in code.
Same evidence can recur	Allows correlated observations.	Natural possibility in retrieval systems; should be measured.
Trust updates are not fully provenance-aware	Allows repeated correlated evidence to accumulate.	Not established by the handoff; requires code audit.
Attacker can influence enough queries/corpus content	Provides the manipulation capability.	Threat-model assumption requiring explicit evaluation.

7. Why This Is Different From the Existing Rings

The existing architecture is largely designed to detect suspicious evidence at different stages: query screening, DRS filtering, risk routing, cohesion/contention analysis, and group-wise counterfactual validation. The Dynamic Trust Store introduces a different class of risk because it changes the system's state based on previous observations.

This means the security boundary is no longer only the current query and retrieved documents. It also includes the integrity of the historical state. An attacker who cannot directly defeat a detector may instead attempt to influence what the detector believes about an evidence source over time.

8. Proposed Attack Experiment

A strong experiment would directly test whether trust can be bootstrapped. Do not merely replay the existing benchmark. Add an adaptive attacker whose objective is to maximize the trust score of a poisoned document/group before launching the final payload.

- Baseline: run the current benchmark without trust persistence.
- Warm-up phase: expose the system to benign-looking queries in which the poisoned evidence repeatedly appears.
- Correlation phase: ensure many observations share the same poisoned source/group.
- Payload phase: issue a target query where the poisoned evidence produces an incorrect answer.

- **Measure:** initial trust, final trust, retrieval rank, routing depth, GWCC activation, final answer accuracy, and ASR.
- **Control:** repeat the same experiment with independent benign sources replacing the repeated poisoned source.

9. Metrics That Should Be Reported

Metric	Purpose
Trust score trajectory	Shows whether an attacker can increase trust before the payload.
Time-to-trust	Number of interactions needed to reach a favorable trust state.
Trust amplification factor	Final trust divided by initial trust under attack.
Source diversity	Number of independent provenance sources behind accumulated evidence.
Effective sample size	Measures how much independent information the historical observations actually contain.
Attack success rate	Measures whether bootstrapped trust improves the final attack.
False-trust rate	Fraction of poisoned sources receiving high trust.
Recovery/decay rate	Measures whether trust can be reduced after contradictory evidence.

10. Recommended Architectural Mitigations

- **Provenance-aware evidence accounting:** do not count repeated observations from the same document, author, source, cluster, or attack campaign as independent confirmations.
- **Trust caps:** place hard upper bounds on how much trust can be gained from historical behavior alone.
- **Conservative trust dynamics:** make trust harder to bootstrap and ensure suspicious evidence can lose trust quickly when contradictory evidence appears.
- **Independent-source requirement:** require corroboration from genuinely distinct sources before assigning high trust.
- **Quarantine for newly trusted evidence:** new documents should not immediately gain strong routing privileges from a small number of observations.
- **Adversarial state testing:** evaluate the trust store itself as an attack surface, not only the retrieval corpus.
- **State rollback / auditability:** retain the evidence responsible for each trust update so suspicious trust trajectories can be reconstructed.
- **Trust isolation:** avoid allowing one document/group's historical score to directly dominate unrelated queries.

11. Important Limitation of This Finding

This report identifies a **potential architectural vulnerability**; it does not claim that the supplied project has already been successfully broken. The handoff does not provide sufficient implementation-level evidence to establish exactly how trust updates affect future retrieval/routing or whether the current implementation already includes protections against correlated evidence.

Therefore, the correct academic claim is: **“The Dynamic Trust Store introduces a potential temporal trust-amplification attack surface that requires provenance-aware and adversarial evaluation.”** It would be premature to claim “OmniGuard is vulnerable” until the code and an attack experiment demonstrate it.

12. Research Significance

If the proposed attack succeeds, it would expose a limitation not captured by ordinary single-query evaluations: a defense can be robust at the per-query level while remaining vulnerable at the state-management level. This would turn the Dynamic Trust Store from a claimed security advantage into a new security boundary that must itself be defended.

If the attack fails because the implementation already enforces independence, provenance, caps, or other safeguards, that is also a valuable result: the project can document the mechanism that prevents trust bootstrapping and demonstrate it experimentally.

13. Bottom Line

The most important architectural question is not “Does OmniGuard remember past behavior?” but “Can an attacker manufacture the past behavior that OmniGuard remembers?”

The supplied handoff makes persistent cross-query trust central to the final defense. That makes the trust-update mechanism an obvious candidate for adversarial security analysis. The proposed flaw is a feedback loop caused by potentially correlated historical evidence. The correct next step is a trust-bootstrap attack experiment and a code-level audit of exactly how trust affects later routing and retrieval.

Source basis: ChatGPT-OmniGuard RAG Handoff-20260830-1140.md, supplied in this conversation. Claims about project architecture and reported ablations are derived from that document; the temporal trust-amplification attack is an analytical inference and is explicitly labeled as such.